

# 22CY702 - Intrusion Detection and Internet Security Lab

Linux Commands, Configuration Files, Snort Rules, Verification Steps, Questions and Explanations

Prepared for practical laboratory study and record preparation

Coverage: Snort installation, alerts and email, HTTP detection, SYN traffic analysis, loopback capture, ICMP spoofing in an isolated lab, iptables, Cisco Packet Tracer tunneling, and custom Snort scan rules.

Safety note: Perform packet-generation experiments only on systems you own or are explicitly authorized to test. Use isolated virtual machines and bounded test traffic.

# How to Use This PDF

Each experiment is arranged as a question followed by the aim, explanation, commands, configuration, verification procedure, expected observation, and important notes. Commands may require adjustment for your Ubuntu version, Snort version, interface name, and lab IP addresses.

## Common interface command:

```
ip -br addr
```

## Common Snort configuration test:

```
sudo snort -T -c /etc/snort/snort.conf
```

Important version note: The configuration syntax in this document primarily follows Snort 2.x conventions, including `snort.conf`, `local.rules`, and HTTP rule options. Snort 3 uses a different configuration architecture, so verify the version before applying the commands.

# Experiment 1: Install Snort and Monitor Live Network Traffic

## Question

Install Snort on a Linux host and configure it to run in packet-capture and intrusion-detection modes to monitor live network traffic.

## Aim

To install Snort, validate its configuration, capture packets, and run it as a Network Intrusion Detection System (NIDS).

## Explanation

Snort is a network security monitoring tool that can capture packets, inspect traffic using rules, and generate alerts when suspicious activity matches a signature. Packet-capture mode is useful for observing traffic, while IDS mode analyzes packets against configured rules.

## Commands

```
sudo apt update
sudo apt install -y snort tcpdump wireshark nmap curl

snort -V
ip -br addr

sudo snort -T -c /etc/snort/snort.conf
```

## Packet-capture mode

```
# Replace enp0s3 with your actual interface
sudo snort -v -i enp0s3
```

## Intrusion-detection mode

```
sudo snort -A console -q \
-c /etc/snort/snort.conf \
-i enp0s3
```

## Verification

```
# In another terminal
ping -c 4 127.0.0.1

# View Snort log files
sudo ls -lh /var/log/snort
```

## Expected observation

Snort starts without configuration errors, observes packets on the selected interface, and displays alerts when traffic matches an enabled rule.

## Viva explanation

Q: What is the difference between packet capture and IDS mode?

A: Packet capture displays or records traffic for analysis. IDS mode compares traffic with detection rules and generates security alerts.

# Experiment 2: Real-Time Intrusion Alerts and Email Notifications

## Question

Configure Snort to generate real-time alerts and automatically send email notifications when suspicious network activity is detected.

## Aim

To configure Snort syslog output and connect alert events to an email notification script.

## Explanation

Snort can write alerts to the console, log files, or syslog. A logging service such as rsyslog can forward selected messages to a program. The program can send an email through an SMTP server. The email mechanism is separate from Snort's detection engine.

## Snort configuration

```
sudo nano /etc/snort/snort.conf

# Add or verify a compatible syslog output line:
output alert_syslog: LOG_AUTH LOG_ALERT
```

## Python mailer

```
sudo apt install -y python3
sudo nano /usr/local/bin/snort-mailer.py

#!/usr/bin/env python3
import sys
import smtplib
from email.message import EmailMessage

SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587
SMTP_USER = "yourgmail@gmail.com"
SMTP_PASS = "YOUR_APP_PASSWORD"
TO_EMAIL = "admin@gmail.com"

alert_body = sys.stdin.read()

if alert_body.strip():
    msg = EmailMessage()
    msg.set_content(alert_body)
    msg["Subject"] = "Snort Alert on localhost"
    msg["From"] = SMTP_USER
    msg["To"] = TO_EMAIL

    try:
        with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
            server.starttls()
            server.login(SMTP_USER, SMTP_PASS)
            server.send_message(msg)
    except Exception as exc:
        print(f"Failed to send email: {exc}")

sudo chmod +x /usr/local/bin/snort-mailer.py
```

## rsyslog configuration

```
sudo nano /etc/rsyslog.d/snort-email.conf

if ($programname == 'snort') then {
    action(type="omprog"
        binary="/usr/local/bin/snort-mailer.py")
}

sudo systemctl restart rsyslog
sudo systemctl status rsyslog
```

## Run and test

```
sudo snort -A console -q \
-c /etc/snort/snort.conf -i lo

# In another terminal
```

```
ping -c 4 127.0.0.1
```

```
# Follow system logs  
sudo journalctl -f
```

## Verification

- Confirm that Snort produces a matching alert.
- Confirm that rsyslog receives the event.
- Confirm that the mailer is invoked and the message reaches the mailbox.

### Security note

Use an SMTP app password or another supported authentication method. Never publish SMTP credentials in source code or GitHub. The exact rsyslog pipeline may require adaptation and testing because Snort output formats and service handling vary by version.

# Experiment 3: Detect Sensitive HTTP GET Requests

## Question

Demonstrate how Snort detects malicious HTTP GET requests that attempt to access sensitive paths such as /admin.

## Aim

To create a local HTTP test server, add a custom Snort rule, and verify that a request for a sensitive path triggers an alert.

## Explanation

HTTP detection rules can inspect the request method and URI. A rule can identify a GET request containing a sensitive path. This is a detection demonstration, not proof that the requested resource is actually protected or compromised.

## Start a local HTTP server

```
mkdir -p ~/snort-http-lab
cd ~/snort-http-lab
python3 -m http.server 8080 --bind 127.0.0.1
```

## Add a Snort 2.x local rule

```
sudo nano /etc/snort/rules/local.rules

alert tcp any any -> any 8080 (
  msg:"Possible Sensitive Path Access";
  flow:to_server,established;
  content:"GET";
  http_method;
  content:"/admin";
  http_uri;
  nocase;
  sid:1000002;
  rev:1;
)
```

## Ensure local rules are included

```
include $RULE_PATH/local.rules
```

## Test and run

```
sudo snort -T -c /etc/snort/snort.conf

sudo snort -A console -q \
  -c /etc/snort/snort.conf -i lo
```

## Generate HTTP requests

```
curl -v http://127.0.0.1:8080/admin
curl -v http://127.0.0.1:8080/index.html
```

## Verification

- The /admin request should match the rule when traffic is decoded as HTTP.
- The /index.html request should not match the /admin content condition.
- Review the console and /var/log/snort for the alert.

## Viva explanation

Q: Why are http\_method and http\_uri used?

A: They restrict content matching to the HTTP method and URI fields, reducing accidental matches in unrelated packet data.

# Experiment 4: Observe TCP SYN Traffic with Wireshark

## Question

Simulate a controlled TCP SYN test against an authorized target and observe the packets using Wireshark.

## Aim

To identify TCP SYN packets, SYN/ACK responses, and handshake behavior in a packet capture.

## Explanation

TCP begins a normal connection with a three-way handshake: SYN, SYN/ACK, and ACK. A large number of incomplete SYN attempts can be associated with SYN-flood behavior, but packet counts alone do not prove service impact.

## Prepare a target

```
# On an isolated target VM
sudo apt update
sudo apt install -y apache2
sudo systemctl start apache2
ip -br addr
```

## Capture with Wireshark

Select the target-facing interface and optionally use the following capture filter:

```
tcp and host 192.168.56.20
```

## Bounded SYN test

```
sudo apt install -y hping3

sudo hping3 -c 100 -i u10000 -S -p 80 192.168.56.20
```

## Wireshark display filters

```
tcp.flags.syn == 1 && tcp.flags.ack == 0

ip.dst == 192.168.56.20 &&
tcp.flags.syn == 1 &&
tcp.flags.ack == 0

ip.src == 192.168.56.20 &&
tcp.flags.syn == 1 &&
tcp.flags.ack == 1
```

## Capture with tcpdump

```
sudo tcpdump -i enp0s3 -nn \
-w syn_test.pcap \
'tcp and host 192.168.56.20'

# Stop with Ctrl+C
tcpdump -nn -r syn_test.pcap
```

## Verification

- Observe SYN-only packets.
- Check whether SYN/ACK responses are returned.
- Inspect packet count and TCP conversations.
- Save the capture as .pcap or .pcapng for the lab record.

Safety note

Use a bounded, low-rate test on an isolated target. Do not use unrestricted flooding or randomized-source options on shared networks.

# Experiment 5: Loopback Packet Capture and Snort Detection

## Question

Capture packets on the loopback interface and demonstrate intrusion detection using Snort.

## Aim

To create loopback aliases, generate local traffic, capture packets, and inspect the resulting pcap.

## Explanation

The loopback interface, normally named lo, is used for communication within the same Linux host. Loopback aliases can represent separate test endpoints without requiring physical network access.

## Create aliases

```
sudo ip addr add 127.0.0.2/8 dev lo
sudo ip addr add 127.0.0.3/8 dev lo
ip -4 addr show dev lo
```

## Capture packets

```
sudo tcpdump -i lo -nn -w loopback_capture.pcap
```

## Run Snort

```
sudo snort -A console -q \
-c /etc/snort/snort.conf -i lo
```

## Generate traffic

```
ping -I 127.0.0.2 -c 4 127.0.0.3

# In another terminal, start a local server
python3 -m http.server 8080 --bind 127.0.0.3

# From another terminal
curl --interface 127.0.0.2 http://127.0.0.3:8080/

sudo nmap -sS -p 22,80,8080 127.0.0.3
```

## Inspect the pcap

```
# Stop tcpdump with Ctrl+C
ls -lh loopback_capture.pcap
tcpdump -r loopback_capture.pcap -nn -tttt
wireshark loopback_capture.pcap &
```

## Wireshark filter

```
ip.src == 127.0.0.2 && ip.dst == 127.0.0.3
```

## Verification

Verify that loopback packets are captured and that enabled Snort rules generate alerts for matching HTTP, ICMP, or scan traffic.

# Experiment 6: ICMP Source-IP Spoofing in an Isolated Loopback Lab

## Question

Demonstrate ICMP echo-request packets with a forged source IP directed to a loopback alias and save the packets as a pcap for analysis.

## Aim

To use Scapy to construct an ICMP packet with a chosen source address, capture the traffic, and inspect the packet fields.

## Explanation

IP spoofing means placing a source IP address in a packet that does not represent the actual sender. In this lab, the demonstration is restricted to a local loopback environment and is used for packet-analysis learning.

## Install packages

```
sudo apt update
sudo apt install -y python3 python3-scapy tcpdump wireshark
python3 -c "from scapy.all import *; print('Scapy OK')"
```

## Capture ICMP packets

```
sudo tcpdump -i lo -c 5 icmp \
-w /tmp/icmp_spoof_loop.pcap
```

## Scapy script

```
nano ~/send_spoof_loop.py

#!/usr/bin/env python3
from scapy.all import IP, ICMP, send, conf

conf.verb = 1
TARGET = "127.0.0.2"
SPOOF_SRC = "1.2.3.4"
IFACE = "lo"

pkt = IP(src=SPOOF_SRC, dst=TARGET) / ICMP(type=8) / b"loopback-spoof"

send(pkt, iface=IFACE, count=5, inter=0.5)
print("Sent 5 spoofed ICMP packets")
```

## Run and inspect

```
sudo python3 ~/send_spoof_loop.py

ls -lh /tmp/icmp_spoof_loop.pcap
sudo tcpdump -nn -v -r /tmp/icmp_spoof_loop.pcap
wireshark /tmp/icmp_spoof_loop.pcap &
```

## Verification

- Filter for ICMP in Wireshark.
- Inspect the source and destination IP fields.
- Confirm ICMP type 8, Echo Request.
- Record whether replies are present; local kernel behavior may vary.

### Important note

Some operating systems apply reverse-path validation, routing, or loopback-specific handling that can affect spoofed packet delivery and replies. The pcap is the primary evidence for this experiment.

# Experiment 7: Configure Linux Firewall Using iptables

## Question

Configure a Linux firewall using iptables and verify filtering rules.

## Aim

To inspect chains, create allow and drop rules, test connectivity, and save or remove firewall rules.

## Explanation

iptables is a command-line interface to Linux packet filtering. The filter table commonly uses INPUT for packets addressed to the host, OUTPUT for packets generated by the host, and FORWARD for routed traffic.

## Install and inspect

```
sudo apt update
sudo apt install -y iptables
iptables -V
sudo iptables -L -n -v --line-numbers
```

## Safe baseline rules

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

```
sudo iptables -A INPUT \
-m conntrack \
--ctstate ESTABLISHED,RELATED \
-j ACCEPT
```

```
sudo iptables -A INPUT \
-p tcp --dport 22 \
-s 10.0.2.7 \
-m conntrack --ctstate NEW \
-j ACCEPT
```

## Logging and default policy

```
sudo iptables -A INPUT \
-m limit --limit 5/min \
-j LOG --log-prefix "iptables-drop: "
```

```
# Apply only in an isolated or recoverable environment
sudo iptables -P INPUT DROP
```

## Inspect and remove rules

```
sudo iptables -L -n -v --line-numbers
sudo iptables -D INPUT <RULE_NUMBER>
```

```
# Flush all rules when appropriate
sudo iptables -F
```

## Save rules

```
sudo apt install -y iptables-persistent
sudo netfilter-persistent save
sudo iptables-save
```

## Verification

- Check rule order and packet counters.
- Test permitted traffic from the authorized source.
- Confirm that unwanted traffic is dropped or logged.
- Do not lock yourself out of a remote server by changing policies without console access or a recovery plan.

# Experiment 8: Configure VPN-Related Tunnel in Cisco Packet Tracer

## Question

Configure a VPN-related topology using Cisco Packet Tracer and verify connectivity between two branch networks.

## Aim

To build a three-router topology, configure LAN/WAN addresses, create tunnel interfaces, and test reachability.

## Explanation

A VPN normally provides a protected communication channel over an untrusted network. The lab manual's shown commands create tunnel interfaces, but they do not by themselves configure IPsec encryption, IKE negotiation, transform sets, or crypto maps. Therefore, the shown configuration should be described as a tunnel demonstration unless IPsec is separately configured and verified.

## Example addressing

| Device  | Interface | Address     |
|---------|-----------|-------------|
| PC0     | LAN       | 192.168.1.2 |
| Router0 | Fa0/0     | 192.168.1.1 |
| Router0 | Fa0/1     | 1.0.0.2     |
| Router2 | Fa0/0     | 1.0.0.1     |
| Router2 | Fa0/1     | 2.0.0.1     |
| Router1 | Fa0/1     | 2.0.0.2     |
| Router1 | Fa0/0     | 192.168.2.1 |
| PC1     | LAN       | 192.168.2.2 |

## Router0 tunnel configuration

```
enable
configure terminal

interface Tunnel1
 ip address 172.16.1.1 255.255.0.0
 tunnel source FastEthernet0/1
 tunnel destination 2.0.0.2
 no shutdown

end
show ip interface brief
```

## Router1 tunnel configuration

```
enable
configure terminal

interface Tunnel2
 ip address 172.16.1.2 255.255.0.0
 tunnel source FastEthernet0/1
 tunnel destination 1.0.0.2
 no shutdown

end
show ip interface brief
```

## Verification commands

```
ping 2.0.0.2
ping 172.16.1.2
show ip route
show interfaces Tunnel1
show interfaces Tunnel2
```

## Verification explanation

The tunnel interfaces should be operational, and the endpoints should be reachable. A genuine IPsec verification would require supported commands such as `show crypto isakmp sa` and `show crypto ipsec sa`, but command support depends on the Packet Tracer device and version.

# Experiment 9: Snort NIDS with Custom Scan Detection Rules

## Question

Deploy Snort as a NIDS on Ubuntu and configure custom rules to detect SYN scan activity, UDP scans, Ping scans, FIN scans, NULL scans, XMAS scans, and generic TCP scans.

## Aim

To create local Snort rules, validate the configuration, generate authorized test traffic, and verify the resulting alerts.

## Explanation

Port scans use different TCP flag combinations or protocols to discover services. Snort rules can identify individual packets, while thresholds such as `detection_filter` can reduce alerts for isolated packets and focus on repeated activity.

## Create the local rules file

```
sudo nano /etc/snort/rules/local.rules
```

## Custom rules

```
alert tcp any any -> $HOME_NET any (
  msg:"[IDS] Possible TCP SYN scan/flood";
  flags:S;
  detection_filter:track by_src, count 20, seconds 5;
  sid:1000001;
  rev:1;
)

alert udp any any -> $HOME_NET any (
  msg:"[IDS] Possible UDP scan";
  detection_filter:track by_src, count 40, seconds 60;
  sid:1000002;
  rev:1;
)

alert icmp any any -> $HOME_NET any (
  msg:"[IDS] ICMP echo request sweep";
  itype:8;
  detection_filter:track by_src, count 10, seconds 60;
  sid:1000003;
  rev:1;
)

alert tcp any any -> $HOME_NET any (
  msg:"[IDS] Possible FIN scan";
  flags:F;
  detection_filter:track by_src, count 5, seconds 60;
  sid:1000004;
  rev:1;
)

alert tcp any any -> $HOME_NET any (
  msg:"[IDS] Possible NULL scan";
  flags:0;
  detection_filter:track by_src, count 5, seconds 60;
  sid:1000005;
  rev:1;
)

alert tcp any any -> $HOME_NET any (
  msg:"[IDS] Possible XMAS scan";
  flags:FPU;
  detection_filter:track by_src, count 5, seconds 60;
  sid:1000006;
  rev:1;
)

alert tcp any any -> $HOME_NET any (
  msg:"[IDS] Generic TCP portscan (SYN probes)";
  flags:S;
  detection_filter:track by_src, count 15, seconds 60;
```

```
sid:1000007;  
rev:1;  
)
```

## Validate and start Snort

```
sudo snort -T -c /etc/snort/snort.conf
```

```
sudo snort -A console -q \  
-c /etc/snort/snort.conf -i enp0s3
```

## Authorized scan tests

```
# Ping test  
ping -c 12 127.0.0.1  
  
# SYN scan  
sudo nmap -sS -p 1-100 127.0.0.1  
  
# TCP connect scan  
sudo nmap -sT -p 1-100 127.0.0.1  
  
# UDP scan  
sudo nmap -sU -p 53,123 127.0.0.1  
  
# FIN scan  
sudo nmap -sF -p 1-100 127.0.0.1  
  
# NULL scan  
sudo nmap -sN -p 1-100 127.0.0.1  
  
# XMAS scan  
sudo nmap -sX -p 1-100 127.0.0.1
```

## Verify alerts

```
sudo ls -lh /var/log/snort  
sudo tail -f /var/log/snort/alert
```

### Important rule note

The rules above are intended for a Snort 2.x-style setup. Some rules overlap, and a SYN rule may alert on legitimate connection attempts. Use thresholds, network variables, and additional context to reduce false positives. Validate syntax and behavior on your installed Snort version.

# Common Troubleshooting Commands

## Find interfaces

```
ip -br addr  
ip link show  
sudo tcpdump -D
```

## Validate Snort

```
sudo snort -T -c /etc/snort/snort.conf  
grep -n "local.rules" /etc/snort/snort.conf
```

## Check running processes

```
ps aux | grep snort  
ps aux | grep tcpdump  
ps aux | grep hping3
```

## Inspect logs

```
sudo find /var/log/snort -maxdepth 1 -type f -ls  
sudo tail -f /var/log/snort/alert
```

## Stop interactive tools

Use Ctrl+C in the terminal running Snort, tcpdump, or another foreground capture command. Check for remaining processes before starting another capture.

# Practical Record Checklist

- Write the experiment question and aim.
- Mention the Linux distribution, Snort version, interface, and IP addresses.
- Include the configuration file or custom rule.
- Attach a screenshot of the command execution.
- Attach the Snort alert, Wireshark view, or pcap evidence.
- Write the verification result.
- Add two to five viva questions and answers.

## Sample result statement

Thus, the experiment was performed in an authorized laboratory environment. The configuration was tested, the required traffic was generated, and the expected packets or Snort alerts were observed where the rule and software configuration matched the test conditions.

## Final reminder

A successful command execution is not sufficient evidence by itself. For a strong lab record, show the configuration, the generated traffic, the captured output, and an explanation of what the output proves.